SERVIÇO NACIONAL DE APRENDIZAGEM INDUSTRIAL
ESCOLA E FACULDADE SENAI “GASPAR RICARDO JUNIOR”

CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

JOÃO ALEXANDRE DA SILVA PEREIRA
LUIS FELIPE CORREIA
MARCO ANTÔNIO
RAFAELA SILVA DA PAZ
TUANNY THOMAZELLI

GESTÃO DE CONSULTAS

Plataforma web para agendamento e gestão de consultas em clínicas médicas

SOROCABA
2026

JOÃO ALEXANDRE DA SILVA PEREIRA
LUIS FELIPE CORREIA
MARCO ANTÔNIO
RAFAELA SILVA DA PAZ
TUANNY THOMAZELLI

GESTÃO DE CONSULTAS

Plataforma web para agendamento e gestão de consultas em clínicas médicas

Trabalho de Conclusão de Curso entregue à Escola e Faculdade SENAI “Gaspar Ricardo Junior”, como requisito para o desenvolvimento acadêmico da formação.

Orientadores: Prof. Esp. Daniel Cintori e Emerson Magalhães.

SOROCABA
2026

Oferecemos este trabalho aos nossos pais, pelo incentivo e apoio; aos professores que confiaram em nosso esforço; e a todos que, de maneira direta ou indireta, colaboraram para a finalização de mais esta jornada.

AGRADECIMENTOS

Somos gratos, em primeiro lugar, a Deus, por nos conceder energia, saúde e discernimento durante toda esta caminhada.

Aos nossos mestres, em especial ao professor Daniel Cintori, por sua dedicação, orientação técnica e constante apoio no decorrer da elaboração deste trabalho. Ao professor Emerson Magalhães, pela paciência, estímulo e pelas contribuições significativas que possibilitaram a finalização deste projeto.

Estendemos nosso agradecimento também ao SENAI, pela infraestrutura de qualidade e pela oportunidade de crescimento pessoal e profissional, e aos profissionais de saúde e recepcionistas que compartilharam conosco a rotina de uma clínica e nos ajudaram a entender suas reais necessidades.

Aos colegas de classe, amigos e familiares, pelo incentivo, compreensão e suporte ao longo desta etapa de aprendizagem.

E, sobretudo, reconhecemos que cada desafio superado foi essencial para nosso amadurecimento e evolução.

RESUMO

Este trabalho apresenta o desenvolvimento da plataforma Gestão de Consultas, um sistema web de agendamento e gestão de consultas para clínicas médicas de pequeno e médio porte, utilizando Node.js com Express no back-end, MongoDB para o gerenciamento de dados e React com TypeScript e Tailwind CSS no front-end. O objetivo principal foi centralizar o agendamento de consultas, a agenda dos profissionais de saúde, o cadastro de pacientes e convênios, a comunicação com os pacientes e a cobrança das consultas particulares, reduzindo faltas, conflitos de horário e o trabalho manual da recepção. O sistema possui quatro perfis de acesso (paciente, recepção, profissional de saúde e administrador), aplica regras que impedem horários duplicados e verificam a cobertura do convênio antes do agendamento, e oferece confirmação de presença, lembretes, fila de espera e pagamento on-line por Pix, cartão e boleto por meio do gateway Stripe. O projeto foi estruturado com metodologia ágil, em ciclos iterativos e incrementais acompanhados em quadro Kanban, com entregas versionadas. Os 41 testes automatizados de integração e os testes de uso apontaram estabilidade na integração entre os módulos e boa experiência de uso. Conclui-se que a solução desenvolvida atende às necessidades funcionais propostas, sendo uma alternativa eficaz para clínicas que desejam modernizar o atendimento de forma simples, segura e acessível.

Palavras-chave: sistemas web; agendamento de consultas; gestão de clínicas; Node.js; MongoDB; React.

LISTA DE FIGURAS

Figura 1 – Logotipo do projeto Gestão de Consultas 11
Figura 2 – Persona do público-alvo 12
Figura 3 – Diagrama de Arquitetura 21
Figura 4 – Diagrama UML de casos de uso 22
Figura 5 – Diagrama de estados da consulta 22
Figura 6 – Modelagem Conceitual 23
Figura 7 – Modelagem Lógica 24
Figura 8 – Modelagem Física 25
Figura 9 – Tela de acesso do sistema 32
Figura 10 – Tela de cadastro do paciente 33

LISTA DE TABELAS

Tabela 1 – Lista de Tecnologias e Ferramentas 14
Tabela 2 – Glossário de Termos 15
Tabela 3 – Cronograma de execução das atividades 16
Tabela 4 – Lista de Requisitos Funcionais 17
Tabela 5 – Lista de Requisitos Não Funcionais 18
Tabela 6 – Regras de Negócio 18
Tabela 7 – Dicionário de dados: usuarios 25
Tabela 8 – Dicionário de dados: pacientes 26
Tabela 9 – Dicionário de dados: profissionais 26
Tabela 10 – Dicionário de dados: consultas 27
Tabela 11 – Dicionário de dados: especialidades e convenios 27
Tabela 12 – Casos de Teste 28
Tabela 13 – Planejamento de testes 29
Tabela 14 – Resultados dos testes 30
Tabela 15 – FAQs e Problemas Comuns 34
Tabela 16 – Documentação da API 36
Tabela 17 – Preço e Condições de Pagamento 40
Tabela 18 – Cronograma de Entrega 41

SUMÁRIO

1 INTRODUÇÃO **10**
2 METODOLOGIA **11**
2.1 Visão do Projeto 11
2.1.1 Nome do projeto *11*
2.1.2 Justificativa *11*
2.1.3 Objetivo *11*
2.1.4 Público-alvo *12*
2.2 Escopo 13
2.2.1 Funcionalidades principais *13*
2.2.2 Limitações e exclusões *13*
2.3 Stakeholders 13
2.3.1 Equipe técnica *13*
2.3.2 Usuários finais *14*
2.3.3 Clientes e patrocinadores *14*
2.4 Metodologias 14
2.5 Tecnologias e Ferramentas 14
2.6 Glossário de termos 15
2.7 Cronograma 16
3 REQUISITOS **17**
3.1 Requisitos Funcionais 17
3.2 Requisitos Não Funcionais 18
3.3 Regras de Negócio 18
3.4 Casos de Uso / Histórias de Usuário 19
4 ARQUITETURA E DESIGN **21**
4.1 Arquitetura de Software 21
4.2 Diagrama UML do sistema 21
4.3 Modelagem de dados 23
4.3.1 Modelagem Conceitual *23*
4.3.2 Modelagem Lógica *24*
4.3.3 Modelagem Física *24*
4.3.4 Dicionário de dados *25*
5 QUALIDADE E TESTES **28**
5.1 Plano de Testes 28
5.2 Casos de Teste 28
5.3 Planejamento de testes 29
5.4 Resultados dos Testes 30

5.5 Testes Automatizados 30
6 MANUAL DO USUÁRIO 32
6.1 Guia de Navegação 32
6.2 FAQs e Problemas Comuns 34
7 MANUAL TÉCNICO 35
7.1 Guia do Desenvolvedor 35
7.1.1 Guia de Instalação 35
7.2 Documentação da API 36
7.2.1 Formato de exemplo de request 37
7.2.2 Resposta esperada 37
7.3 Padrões de Código 37
7.3.1 Backend – JavaScript (Node.js + Express) 37
7.3.2 Frontend – React + TypeScript + Tailwind 38
7.3.3 Boas práticas gerais 38
8 PROPOSTA COMERCIAL 39
8.1 Descrição da Proposta 39
8.2 Escopo Comercial 39
8.2.1 Itens Inclusos 39
8.2.2 Itens Exclusos 40
8.2.3 Preço e Condições de Pagamento 40
8.3 Cronograma de Entrega 41
8.4 Termos e Condições 41
8.5 Validade da Proposta 41
9 CONSIDERAÇÕES FINAIS 42
REFERÊNCIAS 43

1 INTRODUÇÃO

Nos últimos anos, a digitalização de serviços tem se tornado essencial para clínicas médicas que buscam melhorar a experiência do paciente e otimizar seus processos internos. Nesse setor, a marcação de consultas ainda é feita com frequência por telefone, planilhas ou agendas de papel. Esse modelo depende da disponibilidade da recepção, dificulta a visualização dos horários livres e favorece falhas comuns no atendimento, como horários duplicados, agendamentos fora da cobertura do convênio e perda do histórico das alterações. Somam-se a isso as faltas de pacientes que não comparecem e não avisam, deixando ociosos horários que poderiam atender outras pessoas.

Este trabalho apresenta o desenvolvimento da plataforma web Gestão de Consultas, utilizando Node.js com o framework Express no back-end, MongoDB para armazenamento de dados e React com TypeScript e Tailwind CSS no front-end. A proposta visa centralizar em um único lugar o agendamento de consultas, a agenda dos profissionais, o cadastro de pacientes e convênios, a comunicação com o paciente e a cobrança da consulta particular. O paciente passa a ver os horários livres e agendar em poucos passos, enquanto a equipe acompanha a agenda em tempo real. Mecanismos de confirmação de presença, lembretes e fila de espera ajudam a reduzir as faltas e a reaproveitar horários cancelados.

Para a construção do projeto, adotou-se uma metodologia ágil, com desenvolvimento iterativo e incremental, tarefas organizadas em quadro Kanban e entregas versionadas. Essa abordagem possibilitou acompanhar a evolução do sistema de forma contínua, garantindo melhorias constantes e validações ao longo do processo. Os testes automatizados de integração e a validação de uso demonstraram que a aplicação atende aos requisitos funcionais e não funcionais definidos, confirmando sua viabilidade como uma solução eficaz para modernizar a gestão de clínicas médicas.

2 METODOLOGIA

A seguir está descrita a metodologia de desenvolvimento do projeto de software, buscando melhor flexibilidade e distribuição das tarefas entre os integrantes.

2.1 Visão do Projeto

2.1.1 Nome do projeto

Gestão de Consultas – Plataforma web para agendamento e gestão de consultas em clínicas médicas.

Figura 1 – Logotipo do projeto Gestão de Consultas



Fonte: Produção da equipe (2026).

2.1.2 Justificativa

A justificativa para a realização deste projeto está na necessidade crescente de modernização e digitalização dos serviços de saúde. A adoção de ferramentas simples e acessíveis, como plataformas web de agendamento, pode representar um diferencial competitivo para clínicas de pequeno e médio porte, permitindo maior controle sobre a agenda dos profissionais, redução de faltas e de falhas no atendimento e aumento da eficiência operacional da recepção. Além disso, a proposta contribui para a valorização da experiência do paciente, inclusive idosos e pessoas com pouca familiaridade com tecnologia, ao oferecer praticidade, clareza e autonomia para agendar, confirmar, remarcar, cancelar e pagar a consulta por conta própria.

2.1.3 Objetivo

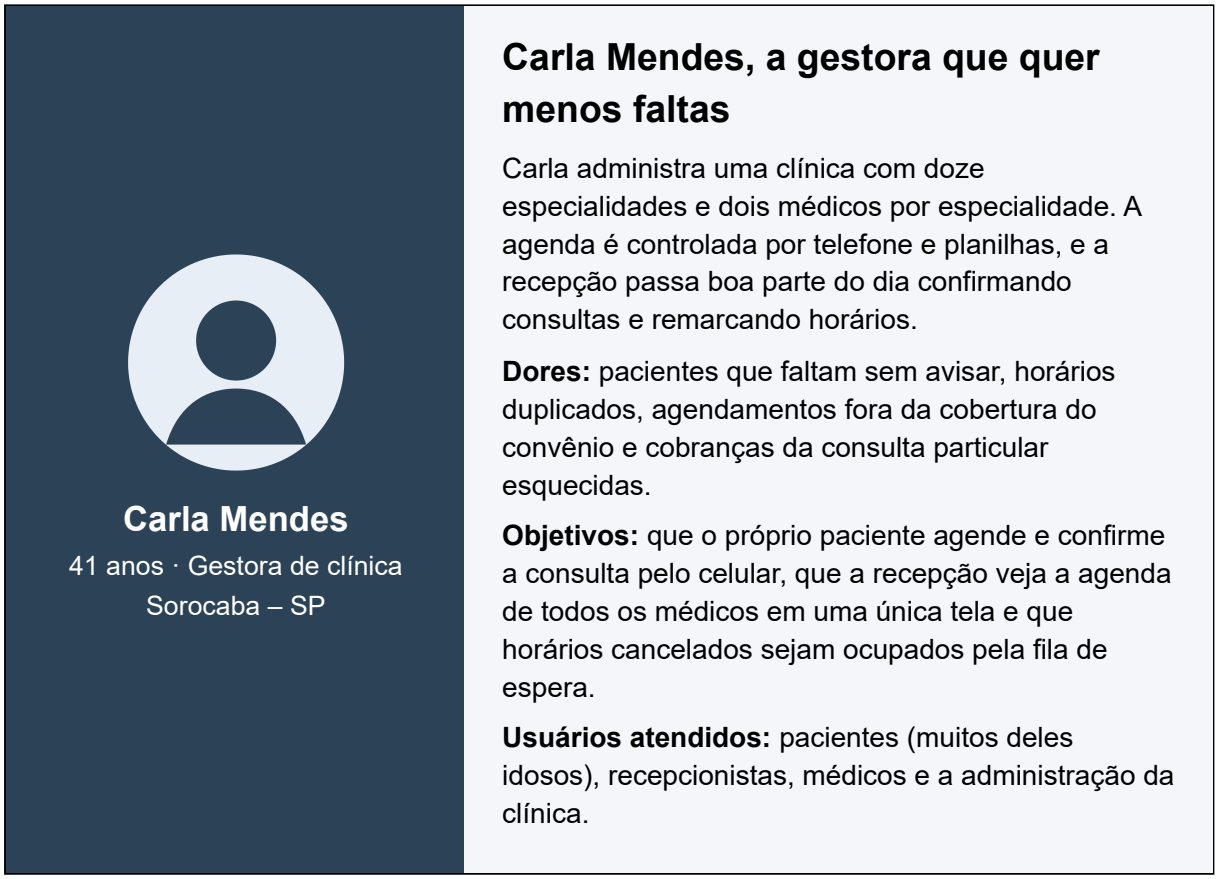
O presente trabalho tem como objetivo geral desenvolver uma plataforma centralizada para o agendamento e a gestão de consultas em uma clínica médica, permitindo que pacientes marquem consultas de forma prática e organizada, ao mesmo tempo em que oferece à clínica maior controle sobre sua agenda e seu fluxo

de atendimento. Para alcançar esse propósito, definem-se como objetivos específicos: modelar os dados de pacientes, profissionais, especialidades, convênios, consultas e notificações; implementar autenticação segura e autorização por perfil de acesso; calcular automaticamente os horários livres de cada profissional, considerando expediente, consultas e bloqueios; validar a cobertura do convênio antes do agendamento; oferecer confirmação de presença, lembretes, fila de espera e notificações para reduzir faltas; integrar um gateway de pagamento para a cobrança da consulta particular; desenvolver uma interface web acessível e responsiva; e garantir a qualidade do código com testes automatizados.

2.1.4 Público-alvo

O público-alvo são clínicas médicas de pequeno e médio porte e as pessoas que fazem parte do seu dia a dia: pacientes de todas as idades, a equipe de recepção, os profissionais de saúde do corpo clínico e os gestores da clínica. Para orientar as decisões de projeto, foi elaborada a persona apresentada na Figura 2.

Figura 2 – Persona do público-alvo



Fonte: Elaborada pelos autores (2026).

2.2 Escopo

2.2.1 Funcionalidades principais

- Projetar e implementar um sistema web de agendamento de consultas, integrando back-end em Node.js com Express, banco de dados MongoDB e front-end em React com TypeScript e Tailwind CSS;
- Desenvolver funcionalidades que permitam ao paciente visualizar horários disponíveis por especialidade e médico e realizar agendamentos de forma prática e intuitiva;
- Oferecer áreas distintas para paciente, recepção, profissional de saúde e administrador, cada uma com as ações pertinentes ao perfil;
- Disponibilizar confirmação de presença, lembretes, notificações e fila de espera para reduzir faltas e reaproveitar horários;
- Permitir o pagamento on-line da consulta particular por Pix, cartão ou boleto dentro do próprio site;
- Garantir o armazenamento seguro e organizado dos dados de pacientes, profissionais, convênios e consultas;
- Validar o funcionamento do sistema por meio de testes automatizados e de uso, verificando sua eficiência, estabilidade e aplicabilidade no contexto de uma clínica real.

2.2.2 Limitações e exclusões

- A versão inicial não possui prontuário eletrônico completo nem integração com sistemas de operadoras de convênio ou com o TISS/ANS;
- O sistema não envia notificações por SMS ou aplicativos de mensagens; os avisos são feitos pelo portal e por e-mail;
- Não há teleconsulta (atendimento por vídeo) nesta versão;
- O sistema se concentra no agendamento e na gestão das consultas, sem incluir controle de estoque, folha de pagamento ou contabilidade da clínica.

2.3 Stakeholders

2.3.1 Equipe técnica

- João Alexandre – Gestor, documentador e desenvolvedor back-end;
- Luis Felipe Correia – Desenvolvedor back-end e banco de dados;
- Marco Antônio – Gestor e desenvolvedor front-end;

- Rafaela Silva da Paz – UX Designer e desenvolvedora front-end;
- Tuanny Thomazelli – UX Designer, pesquisadora e analista de testes.

2.3.2 Usuários finais

- Pacientes da clínica, que agendam e acompanham as próprias consultas;
- Recepcionistas, que administram a agenda, os cadastros e a fila de espera;
- Profissionais de saúde, que consultam a agenda e registram os atendimentos;
- Administradores, que configuram a clínica e acompanham os indicadores.

2.3.3 Clientes e patrocinadores

Clínica médica parceira (cliente), representada pela gestora da clínica, e a Escola e Faculdade SENAI “Gaspar Ricardo Junior”, como instituição de ensino que acompanha o projeto.

2.4 Metodologias

A metodologia utilizada consiste em uma pesquisa prática, com enfoque aplicado e caráter experimental. O desenvolvimento do sistema foi estruturado em três fases principais: (1) levantamento das necessidades da clínica e análise das dificuldades enfrentadas no agendamento por telefone e planilhas; (2) desenvolvimento da plataforma web, com foco em usabilidade, acessibilidade, autonomia do paciente e eficiência no gerenciamento da agenda; e (3) realização de testes e validação da aplicação em ambiente de nuvem, garantindo que o sistema atenda às demandas da clínica.

O trabalho seguiu uma abordagem iterativa e incremental. Cada funcionalidade foi registrada como uma issue com código padronizado (por exemplo, `[CON-03-BE] Remarcação de consulta`), agrupada em épicos por módulo e acompanhada em quadro Kanban no GitHub Projects, com as colunas Backlog, To do, In progress, In review e Done. O controle de versão adotou o modelo Git Flow com revisão por pull request, versionamento semântico (MAIOR.MENOR.CORREÇÃO) e mensagens de commit no formato `tipo: descrição`.

2.5 Tecnologias e Ferramentas

Tabela 1 – Lista de Tecnologias e Ferramentas

Categoria	Tecnologia/Ferramenta	Finalidade
Organização e Planejamento	GitHub Projects	Quadro Kanban com issues, épicos, labels e milestones por versão

Prototipação	Figma / Lovable	Protótipos e wireframes das telas antes da implementação
Edição de código	Visual Studio Code	Edição do código do front-end e do back-end
Back-end	Node.js 20+ e Express 5	API REST com rotas, middlewares e regras de negócio
Front-end	React 19, TypeScript, TanStack Start, Tailwind CSS 4, Radix UI	Interface web responsiva e acessível
Banco de dados	MongoDB (Mongoose) / MongoDB Atlas	Armazenamento dos dados da clínica em banco gerenciado
Autenticação	JWT e bcrypt	Sessão por token e armazenamento seguro das senhas
Pagamentos	Stripe	Checkout embutido com Pix, cartão e boleto
E-mail	Brevo / Nodemailer	Códigos de redefinição de senha, avisos e cobranças
Teste de API	node:test, Supertest e Postman	Testes automatizados de integração e testes manuais das rotas
Controle de Versões	Git e GitHub	Git Flow, pull requests e releases
Hospedagem	Render e Vercel	Publicação da API e da interface web
Documentação	Word / Markdown	Documentação acadêmica, técnica e do usuário

Fonte: Elaborada pelos autores (2026).

2.6 Glossário de termos

Tabela 2 – Glossário de Termos

Termo	Definição
Front-end	Interface com que o paciente e a equipe interagem; desenvolvida com React, TypeScript e Tailwind CSS
Back-end	Servidor que aplica as regras de negócio e controla as permissões; desenvolvido com Node.js e Express
Banco de dados	Local onde ficam armazenados os dados da clínica; utilizamos MongoDB
API REST	Conjunto de rotas que conecta o front-end ao back-end, trocando dados em JSON sobre HTTPS
JWT	JSON Web Token: token assinado que identifica o usuário logado em cada requisição
Convênio	Plano de saúde do paciente, que define quais especialidades são cobertas; “Particular” indica pagamento direto
Fila de espera	Lista de pacientes aguardando vaga em uma especialidade, convidados quando um horário é liberado

Bloqueio	Horário ou período em que o profissional não atende (férias, congresso, reunião)
Webhook	Aviso automático enviado pelo Stripe à API quando um pagamento é concluído
LGPD	Lei Geral de Proteção de Dados (Lei nº 13.709/2018)

Fonte: Elaborada pelos autores (2026).

2.7 Cronograma

Tabela 3 – Cronograma de execução das atividades

Atividades	Data	Status
Reunião inicial com a clínica / Entrevista	03/08 a 05/08	FINALIZADA
Definição dos requisitos e levantamento das necessidades	05/08 a 10/08	FINALIZADA
Criação do escopo e definição do MVP	10/08 a 14/08	FINALIZADA
Elaboração da proposta técnica e comercial	14/08 a 17/08	FINALIZADA
Criação de protótipos e fluxogramas (UX/UI)	17/08 a 22/08	FINALIZADA
Validação dos protótipos com a clínica	24/ago	FINALIZADA
Configuração do ambiente e do quadro Kanban	25/08 a 26/08	FINALIZADA
Desenvolvimento da API (Node.js + Express – estrutura base)	27/08 a 25/09	FINALIZADA
Modelagem e criação do banco de dados (MongoDB)	27/08 a 10/09	FINALIZADA
Desenvolvimento do front-end (React, TypeScript, Tailwind – layout base)	08/09 a 05/10	FINALIZADA
Integração front-end + back-end (autenticação e perfis)	21/09 a 28/09	FINALIZADA
Implementação do módulo de agenda e consultas	28/09 a 09/10	FINALIZADA
Fila de espera, notificações, atestados e receitas	09/10 a 16/10	FINALIZADA
Pagamento on-line da consulta particular (Stripe)	16/10 a 23/10	FINALIZADA
Testes automatizados de integração	12/10 a 26/10	FINALIZADA
Ajustes técnicos e refatoração	26/10 a 30/10	FINALIZADA
Publicação em nuvem (Render, Vercel, MongoDB Atlas)	28/10 a 31/10	FINALIZADA
Testes com usuários (clínica/local)	02/11 a 06/11	FINALIZADA
Documentação técnica e manual do usuário	02/11 a 10/11	EM ANDAMENTO
Preparação da apresentação (slides + roteiro)	10/11 a 13/11	PENDENTE
Entrega técnica final (sistema + docs + mídias)	13/nov	PENDENTE
Semana de revisão e testes finais	16/11 a 20/11	PENDENTE
Submissão da versão final para a banca	23/nov	PENDENTE
Apresentação oficial do projeto (banca)	04/dez	PENDENTE

Fonte: Elaborada pelos autores (2026).

3 REQUISITOS

Esta seção define o que o sistema deve fazer (funcionalidades) e como ele deve se comportar (qualidade, desempenho, restrições). Serve como base para o desenvolvimento, testes e validação do sistema.

3.1 Requisitos Funcionais

Tabela 4 – Lista de Requisitos Funcionais

Código	Requisito
RF001	Autenticar usuários por e-mail e senha, com sessão por token JWT.
RF002	Permitir o cadastro público do paciente, com validação de CPF e escolha do convênio.
RF003	Permitir que cada usuário edite os próprios dados e troque a senha.
RF004	Redefinir a senha por código de 6 dígitos enviado por e-mail, com validade de 15 minutos, limite de tentativas e confirmação de CPF.
RF005	Manter o cadastro de pacientes, com ativação e desativação que preservam o histórico.
RF006	Manter o cadastro de profissionais, com especialidade, CRM, contato, expediente semanal e acesso próprio.
RF007	Manter os catálogos de especialidades e convênios, incluindo as especialidades cobertas por cada convênio.
RF008	Registrar bloqueios de agenda por horário ou por período.
RF009	Calcular os horários livres de cada profissional em intervalos de 30 minutos.
RF010	Verificar a elegibilidade do paciente (situação cadastral e cobertura do convênio) antes do agendamento.
RF011	Agendar, confirmar, remarcar e cancelar consultas, com código sequencial e canal de origem.
RF012	Registrar a consulta como realizada ou como não comparecimento.
RF013	Manter o histórico de alterações de cada consulta.
RF014	Gerar a cobrança da consulta particular após a realização e permitir o pagamento on-line por Pix, cartão ou boleto.
RF015	Permitir que a recepção registre pagamentos feitos presencialmente.
RF016	Enviar notificações por perfil, por paciente ou por profissional, com controle de leitura.
RF017	Enviar lembretes de consulta individuais ou em lote.
RF018	Manter a fila de espera por especialidade, com prioridade e convite quando um horário for liberado.
RF019	Permitir que o profissional envie atestados e receitas ao paciente.
RF020	Disponibilizar indicadores de consultas, presença, faltas, cancelamentos e ocupação da agenda.

RF021	Manter as configurações da clínica, como dados cadastrais e valor da consulta particular.

Fonte: Elaborada pelos autores (2026).

3.2 Requisitos Não Funcionais

Tabela 5 – Lista de Requisitos Não Funcionais

Código	Categoria	Requisito
RNF001	Segurança	Senhas armazenadas com hash bcrypt; rotas protegidas por token JWT e por perfil.
RNF002	Privacidade	Cada perfil acessa somente os dados que lhe dizem respeito, conforme a LGPD (Lei nº 13.709/2018).
RNF003	Pagamentos	Dados de cartão e de pagamento não trafegam nem são armazenados no servidor da clínica.
RNF004	Interoperabilidade	Comunicação em JSON sobre HTTPS, no estilo REST.
RNF005	Disponibilidade	API publicada em nuvem (Render) e banco gerenciado (MongoDB Atlas).
RNF006	Manutenibilidade	Código organizado em camadas (rotas, controladores, serviços e modelos).
RNF007	Qualidade	Testes automatizados de integração cobrindo os fluxos principais.
RNF008	Rastreabilidade	Alterações de consultas registradas com data e descrição.
RNF009	Usabilidade e acessibilidade	Interface responsiva (celular, tablet e computador), legível e navegável por teclado e leitores de tela.

Fonte: Elaborada pelos autores (2026).

3.3 Regras de Negócio

Tabela 6 – Regras de Negócio

Código	Regra
RN01	Os horários são oferecidos em intervalos de 30 minutos, dentro do expediente e sem consultas ativas ou bloqueios.
RN02	No dia corrente, só são oferecidos horários posteriores à hora atual.
RN03	O agendamento exige paciente ativo, especialidade ativa e coberta pelo convênio e, para quem tem convênio, carteirinha válida no dia da consulta.
RN04	O cancelamento libera o horário imediatamente.
RN05	Após a remarcação, a confirmação volta para “Sem resposta” até o paciente confirmar a nova data.
RN06	Toda consulta nasce aguardando confirmação; só o próprio paciente confirma a presença.
RN07	A cobrança é gerada somente para pacientes do convênio Particular e somente após a consulta realizada.

RN08	O valor cobrado é definido no servidor; pagamento com valor inferior ao cobrado não é aceito.
RN09	Pacientes, profissionais e especialidades desativados mantêm o histórico e deixam de aceitar agendamentos.
RN10	Um profissional só pode ser excluído se não tiver consultas registradas.
RN11	Sem resultado registrado, a consulta é encerrada automaticamente 1 hora após o horário: confirmada vira “Realizada”; sem confirmação vira “Não compareceu”.
RN12	Consulta encerrada não pode ser cancelada nem remarcada.

Fonte: Elaborada pelos autores (2026).

3.4 Casos de Uso / Histórias de Usuário

ID: HU01

Como gestora da clínica,

Quero que o próprio paciente agende e confirme a consulta pela internet,

Para que a recepção tenha menos trabalho manual e a clínica tenha menos faltas.

Caso de Uso: Agendar consulta.

Ator: Paciente.

Fluxo principal:

1. O paciente acessa o sistema e faz login.
2. Escolhe a especialidade e o médico.
3. Seleciona a data e um dos horários livres exibidos.
4. O sistema verifica a elegibilidade (cadastro ativo e cobertura do convênio) e registra a consulta como “Agendada”.
5. O paciente recebe o pedido de confirmação pelo portal e por e-mail e confirma a presença.

Fluxo alternativo: 3a. Se não houver horário disponível, o sistema informa a indisponibilidade e o paciente pode entrar na fila de espera. 4a. Se o convênio não cobrir a especialidade, o sistema explica o motivo e impede o agendamento.

ID: HU02

Como recepcionista,

Quero ver a agenda de todos os médicos e ocupar horários cancelados com a fila de espera,

Para que nenhum horário fique ocioso.

Caso de Uso: Gerenciar fila de espera.

Ator: Recepção.

Fluxo principal:

1. Um paciente cancela a consulta e o horário é liberado.
2. A recepção abre a fila de espera da especialidade, ordenada por prioridade.
3. Convida o próximo paciente, que recebe a notificação.
4. A recepção agenda o paciente no horário liberado.

ID: HU03

Como paciente particular,

Quero pagar a consulta pelo próprio site,

Para que eu não precise ir até a recepção.

Caso de Uso: Pagar consulta particular. **Ator:** Paciente.

Fluxo principal: após a consulta ser registrada como realizada, o paciente recebe a notificação de pagamento, clica em “Pagar”, escolhe Pix, cartão ou boleto no Checkout embutido do Stripe e recebe o comprovante.

4 ARQUITETURA E DESIGN

Esta seção descreve como o sistema é estruturado internamente, suas camadas, componentes, tecnologias e como eles se relacionam. Ajuda desenvolvedores e arquitetos a compreenderem o funcionamento técnico e a manterem a consistência do projeto.

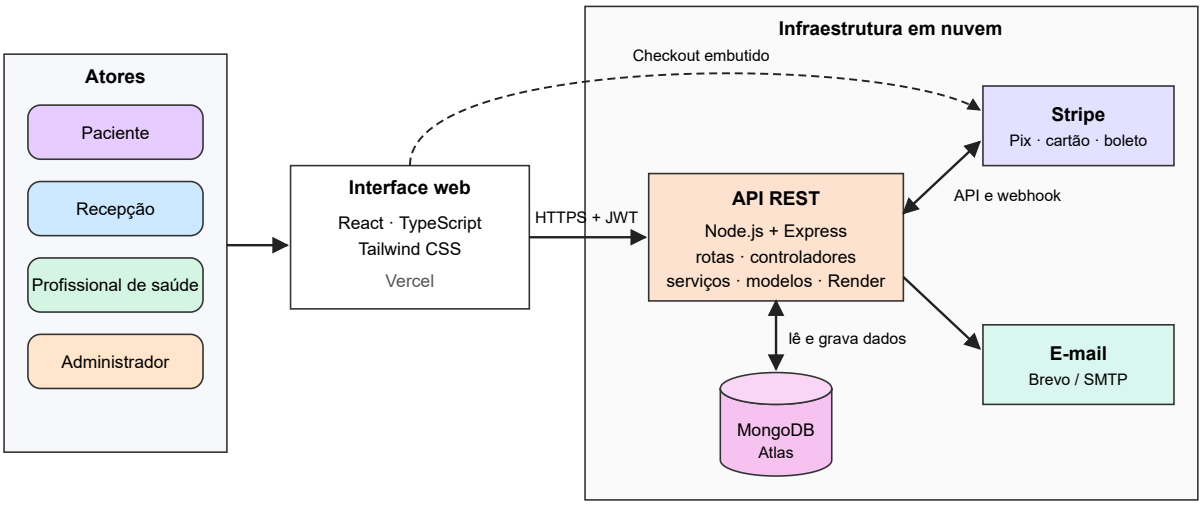
4.1 Arquitetura de Software

Estilo arquitetural: cliente-servidor em três camadas, com API REST organizada em camadas (rotas, controladores, serviços e modelos).

Descrição: o sistema é dividido em três camadas principais, complementadas por serviços externos de e-mail e pagamento:

- **Cliente (interface web):** aplicação React publicada na Vercel, que exibe as telas de cada perfil e consome a API.
- **Servidor de aplicação (API REST):** Node.js com Express publicado no Render; as rotas definem caminhos e permissões, os controladores tratam as requisições, os serviços concentram regras e integrações e os modelos descrevem as coleções.
- **Banco de dados:** MongoDB Atlas, acessado pela biblioteca Mongoose.

Figura 3 – Diagrama de Arquitetura



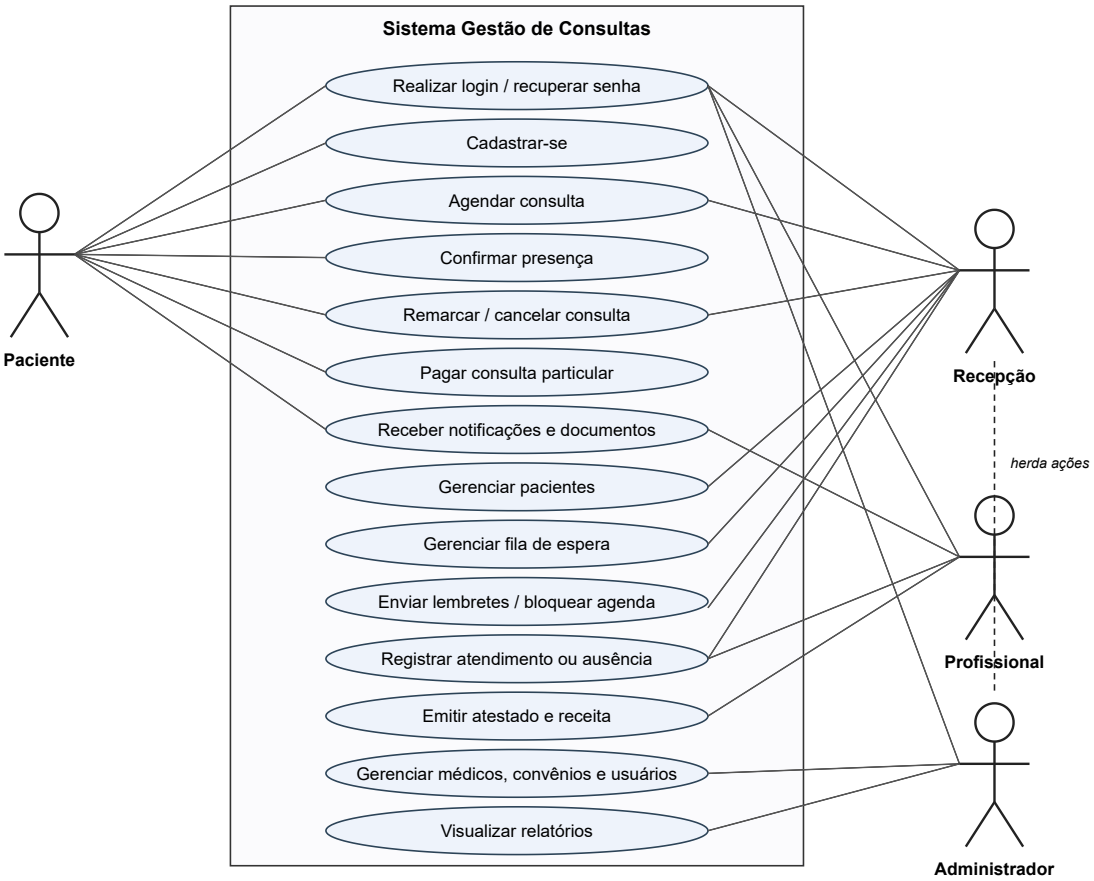
Fonte: Elaborada pelos autores (2026).

4.2 Diagrama UML do sistema

O diagrama de casos de uso da Figura 4 apresenta os quatro atores do sistema e as principais funcionalidades disponíveis para cada um. O administrador

possui todas as ações da recepção, além das ações exclusivas indicadas.

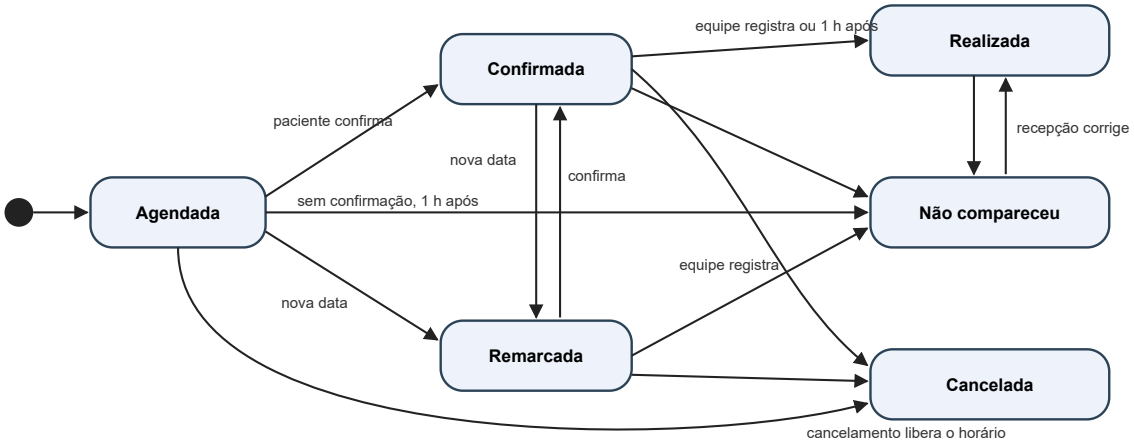
Figura 4 – Diagrama UML de casos de uso



Fonte: Elaborada pelos autores (2026).

O ciclo de vida de uma consulta, com os estados possíveis e as transições permitidas pelas regras de negócio, é apresentado no diagrama de estados da Figura 5.

Figura 5 – Diagrama de estados da consulta



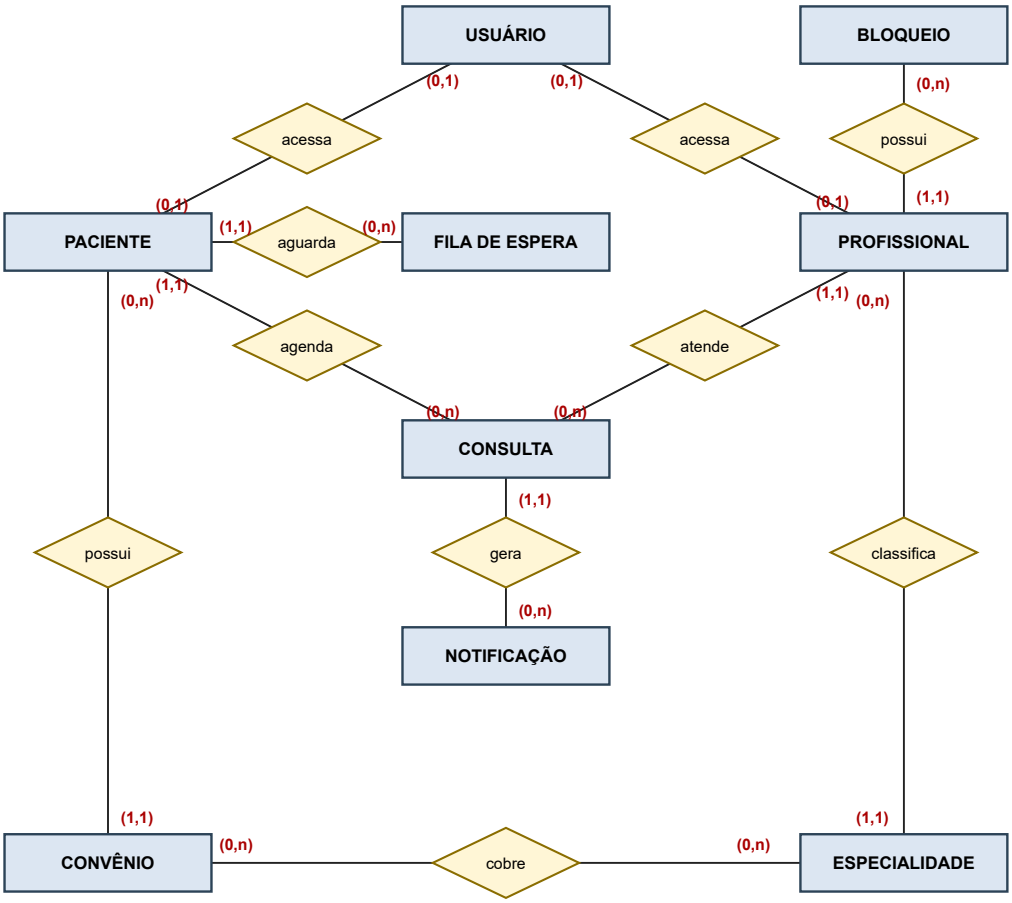
Fonte: Elaborada pelos autores (2026).

4.3 Modelagem de dados

A modelagem de dados foi essencial neste projeto para organizar, de forma estruturada, os principais elementos do sistema de gestão da clínica, como pacientes, profissionais de saúde, especialidades, convênios, consultas, bloqueios de agenda, fila de espera e notificações. Esse processo envolveu a criação dos modelos conceitual, lógico e físico, possibilitando a construção de um banco de dados orientado a documentos no MongoDB, com validação de esquema pela biblioteca Mongoose, integridade das referências e clareza nas relações entre as entidades. A modelagem garantiu que o sistema armazenasse corretamente as informações e permitisse consultas rápidas, precisas e seguras, facilitando a gestão eficiente da agenda e das consultas da clínica.

4.3.1 Modelagem Conceitual

Figura 6 – Modelagem Conceitual

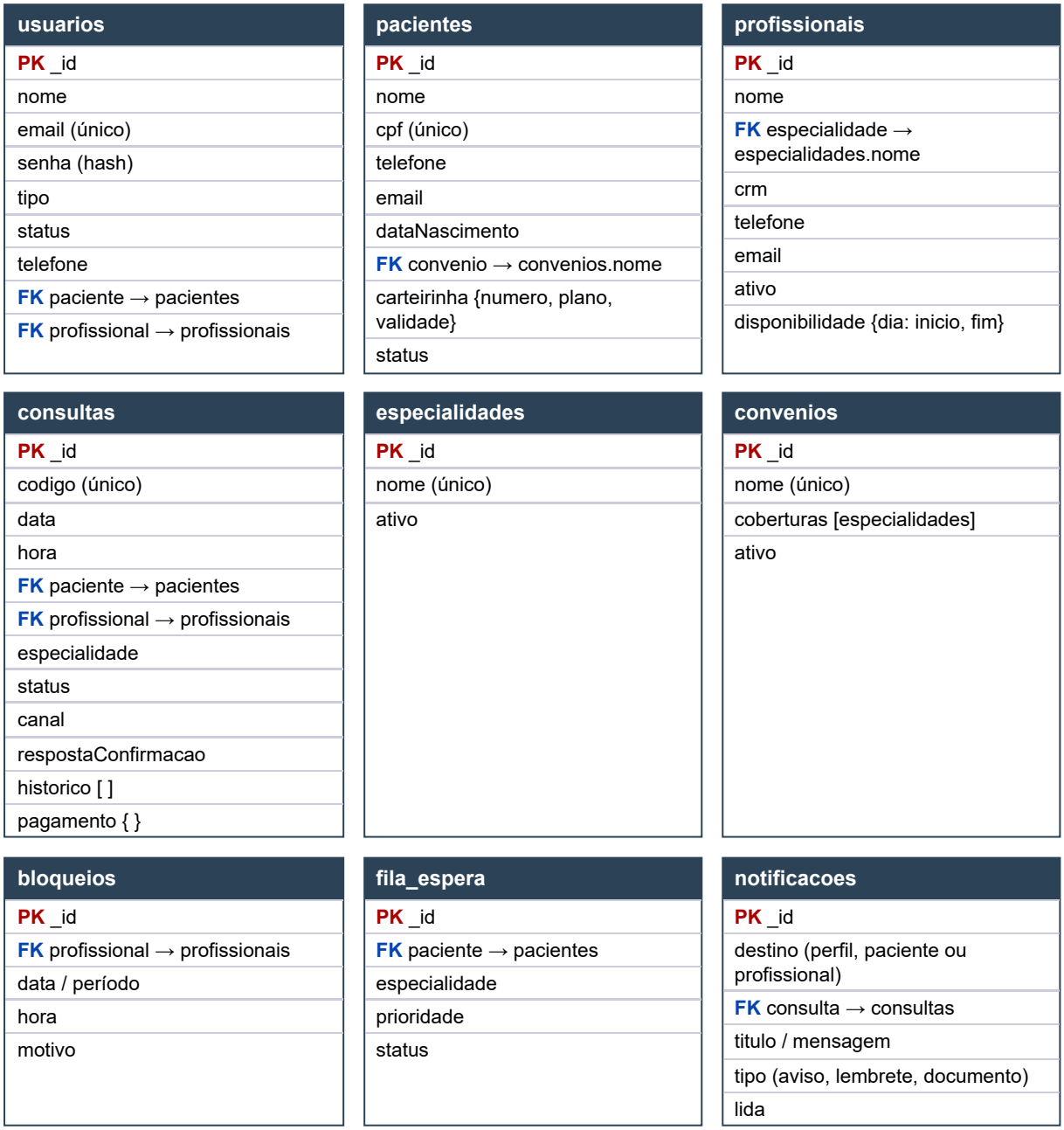


Fonte: Elaborada pelos autores (2026).

4.3.2 Modelagem Lógica

No modelo lógico, cada entidade foi convertida em uma coleção do MongoDB. As chaves primárias (PK) correspondem ao campo `_id` gerado automaticamente, e as chaves estrangeiras (FK) são referências por `ObjectId` entre documentos.

Figura 7 – Modelagem Lógica



Fonte: Elaborada pelos autores (2026).

4.3.3 Modelagem Física

O modelo físico corresponde aos esquemas Mongoose implementados na API, que definem os tipos, as validações e os valores padrão de cada coleção. A

Figura 8 apresenta, de forma resumida, o esquema das coleções `pacientes` e `consultas`.

Figura 8 – Modelagem Física

```
// src/models/Paciente.js
const pacienteSchema = new mongoose.Schema({
  nome:           { type: String, required: [true, "Informe o nome do paciente"], trim:
true },
  cpf:            { type: String, required: [true, "Informe o CPF"], unique: true },
  telefone:       { type: String, trim: true, default: "" },
  email:          { type: String, required: [true, "Informe o e-mail"], lowercase: true
},
  dataNascimento: { type: String, default: "" },
  convenio:       { type: String, default: "Particular" },
  carteirinha:    { numero: String, validade: String, plano: String },
  status:         { type: String, enum: ["Ativo", "Inativo"], default: "Ativo" }
}, { timestamps: true });

// src/models/Consulta.js
const consultaSchema = new mongoose.Schema({
  codigo:        { type: String, required: true, unique: true },
  data:          { type: String, required: true, match: /^\d{4}-\d{2}-\d{2}$/ },
  hora:          { type: String, required: true, match: /^\d{2}:\d{2}$/ },
  paciente:      { type: ObjectId, ref: "Paciente", required: true },
  profissional:  { type: ObjectId, ref: "Profissional", required: true },
  especialidade: { type: String, required: true },
  status:        { type: String, enum: STATUS, default: "Agendada" },
  canal:         { type: String, enum: ["Paciente", "Recepção"], default: "Recepção" },
  respostaConfirmacao: { type: String, enum: ["Confirmou", "Cancelou", "Sem resposta"]
},
  historico:     [{ em: String, texto: String }],
  pagamento: {
    status: { type: String, enum: ["Pendente", "Pago", null] },
    valor: Number, metodo: String, gateway: { type: String, enum: ["", "Stripe",
"Recepção"] },
    sessaoId: String, transacaoId: String, pagoEm: Date
  }
}, { timestamps: true });
```

Fonte: Código-fonte do projeto (2026).

4.3.4 Dicionário de dados

O dicionário de dados descreve cada campo das principais coleções, com seu tipo, tamanho e restrições (PK – chave primária; FK – chave estrangeira; NN – não nulo; UQ – único).

Tabela 7 – Dicionário de dados: usuarios

Nome	Descrição	Tipo de dados	Tamanho	Restrição
_id	Identificador do usuário	ObjectId	12 bytes	PK
nome	Nome do usuário	String	100	NN
email	E-mail de acesso	String	100	NN, UQ

senha	Senha criptografada (hash bcrypt)	String	60	NN
tipo	Perfil: paciente, recepção, profissional ou administrador	String (enum)	–	NN
status	Ativo, Inativo ou Pendente	String (enum)	–	NN
telefone	Telefone de contato	String	15	–
paciente	Ficha de paciente vinculada	ObjectId	12 bytes	FK
profissional	Cadastro de profissional vinculado	ObjectId	12 bytes	FK

Fonte: Elaborada pelos autores (2026).

Tabela 8 – Dicionário de dados: pacientes

Nome	Descrição	Tipo de dados	Tamanho	Restrição
_id	Identificador do paciente	ObjectId	12 bytes	PK
nome	Nome completo do paciente	String	100	NN
cpf	CPF validado no cadastro	String	11	NN, UQ
telefone	Telefone do paciente	String	15	–
email	E-mail do paciente	String	100	NN
dataNascimento	Data de nascimento	String (data)	10	–
convenio	Convênio do paciente (padrão “Particular”)	String	60	NN
carteirinha	Número, plano e validade da carteirinha	Objeto	–	NN se houver convênio
status	Ativo ou Inativo	String (enum)	–	NN

Fonte: Elaborada pelos autores (2026).

Tabela 9 – Dicionário de dados: profissionais

Nome	Descrição	Tipo de dados	Tamanho	Restrição
_id	Identificador do profissional	ObjectId	12 bytes	PK
nome	Nome do médico	String	100	NN
especialidade	Especialidade de atuação	String	60	NN, FK
crm	Registro no Conselho Regional de Medicina	String	20	NN
telefone	Telefone de contato	String	15	–
email	E-mail do profissional	String	100	NN
ativo	Indica se aceita agendamentos	Boolean	–	NN

disponibilidade	Expediente semanal (início e fim por dia)	Map	7 dias	NN

Fonte: Elaborada pelos autores (2026).

Tabela 10 – Dicionário de dados: consultas

Nome	Descrição	Tipo de dados	Tamanho	Restrição
_id	Identificador da consulta	ObjectId	12 bytes	PK
codigo	Código sequencial da consulta	String	12	NN, UQ
data	Data no formato AAAA-MM-DD	String	10	NN
hora	Horário no formato HH:MM	String	5	NN
paciente	Paciente atendido	ObjectId	12 bytes	NN, FK
profissional	Médico responsável	ObjectId	12 bytes	NN, FK
especialidade	Especialidade da consulta	String	60	NN
status	Agendada, Confirmada, Remarcada, Realizada, Cancelada ou Não compareceu	String (enum)	–	NN
canal	Origem do agendamento (Paciente ou Recepção)	String (enum)	–	NN
historico	Registros de alteração com data e descrição	Array	–	–
pagamento	Status, valor, método, gateway e comprovante	Objeto	–	–

Fonte: Elaborada pelos autores (2026).

Tabela 11 – Dicionário de dados: especialidades e convenios

Coleção	Nome	Descrição	Tipo de dados	Restrição
especialidades	_id	Identificador da especialidade	ObjectId	PK
	nome	Nome da especialidade	String	NN, UQ
	ativo	Indica se aceita agendamentos	Boolean	NN
convenios	_id	Identificador do convênio	ObjectId	PK
	nome	Nome do convênio	String	NN, UQ
	coberturas	Especialidades cobertas pelo convênio	Array de String	–
	ativo	Indica se o convênio está ativo	Boolean	NN

Fonte: Elaborada pelos autores (2026).

5 QUALIDADE E TESTES

Esta seção documenta as estratégias de verificação e validação usadas para garantir que o software funcione corretamente, atenda aos requisitos e tenha qualidade técnica e funcional.

5.1 Plano de Testes

Plano de Testes – Gestão de Consultas

Objetivo: garantir que as funcionalidades e os dados principais operem corretamente para os quatro perfis de acesso.

Escopo: cadastro, login, recuperação de senha, agendamento, confirmação, remarcação, cancelamento, bloqueios de agenda, fila de espera, notificações, pagamentos e permissões por perfil.

Responsáveis: equipe de QA e desenvolvedores.

Ambiente: banco de dados de teste separado, com Stripe e e-mail simulados (nenhuma chamada sai para a internet durante os testes automatizados).

Critérios de entrada: funcionalidade implementada, revisada por pull request e integrada à branch `develop`.

Critérios de saída: 100% dos testes automatizados e manuais aprovados, sem erros críticos.

5.2 Casos de Teste

Tabela 12 – Casos de Teste

ID	RF	Descrição	Pré-condição	Passos	Resultado esperado
CT-001	RF001	Validar login do usuário	Usuário cadastrado e não logado	1. Acessar o site; 2. Informar e-mail; 3. Informar a senha; 4. Clicar em “Entrar”.	Usuário direcionado para a área do seu perfil.
CT-002	RF001	Recusar senha incorreta	Usuário cadastrado	1. Informar e-mail válido; 2. Informar senha errada; 3. Clicar em “Entrar”.	Mensagem de credenciais inválidas; acesso negado.

CT-003	RF002	Cadastro de paciente	Pessoa sem cadastro	1. Clicar em “Cadastre-se”; 2. Preencher nome, CPF, telefone, e-mail, senha, nascimento e convênio; 3. Clicar em “Criar conta”.	Conta criada e paciente já entra no sistema.
CT-004	RF002	Recusar CPF inválido ou e-mail repetido	E-mail já cadastrado	1. Abrir o cadastro; 2. Informar CPF inválido ou e-mail já usado; 3. Enviar.	Sistema informa o erro e não cria a conta.
CT-005	RF009 RF011	Agendar consulta	Paciente logado e ativo	1. Abrir “Agendar”; 2. Escolher especialidade e médico; 3. Escolher data e horário; 4. Confirmar.	Consulta criada como “Agendada” e o horário sai da lista de livres.
CT-006	RF011	Recusar horário já ocupado	Horário ocupado por outra consulta	1. Tentar agendar no mesmo médico, data e hora.	Sistema recusa por conflito de horário.
CT-007	RF010	Bloquear convênio sem cobertura	Convênio não cobre a especialidade	1. Paciente tenta agendar a especialidade não coberta.	Sistema explica o motivo e impede o agendamento.
CT-008	RF011	Cancelar consulta	Consulta agendada antes do horário	1. Abrir a consulta; 2. Clicar em “Cancelar”; 3. Confirmar.	Consulta cancelada e horário liberado imediatamente.
CT-009	RF014	Pagar consulta particular	Consulta particular realizada	1. Abrir a notificação de pagamento; 2. Clicar em “Pagar”; 3. Pagar com Pix ou cartão.	Pagamento confirmado pelo webhook e comprovante exibido.
CT-010	RNF002	Isolamento de dados por perfil	Paciente logado	1. Tentar acessar consultas de outro paciente ou a fila de espera.	Acesso negado; paciente vê apenas os próprios dados.

Fonte: Elaborada pelos autores (2026).

5.3 Planejamento de testes

Tabela 13 – Planejamento de testes

Tipo de teste	Ferramenta	Responsável	Frequência
Testes de integração da API	node:test + Supertest	Desenvolvedores back-end	A cada mudança relevante e antes de cada release
Testes manuais das rotas	Postman	Desenvolvedores e QA	Durante o desenvolvimento de cada funcionalidade
Testes de interface e usabilidade	Navegador (desktop e celular)	UX Designers e QA	Ao final de cada ciclo de entrega

Testes de aceitação	Ambiente publicado (Render/Vercel)	Equipe e clínica parceira	Antes da entrega final
Revisão de código	Pull requests no GitHub	Toda a equipe	A cada pull request

Fonte: Elaborada pelos autores (2026).

5.4 Resultados dos Testes

Tabela 14 – Resultados dos testes

Área	Cenários verificados	Resultado
Autenticação	Senha incorreta, rotas sem token, cadastro de paciente, CPF inválido, e-mail repetido, redefinição de senha por código	Aprovado
Permissões	Paciente vê só as próprias consultas; paciente não acessa a fila de espera; recepção não gerencia usuários internos	Aprovado
Agenda	Horários livres de 30 em 30 minutos, bloqueios de período e horários que já passaram	Aprovado
Consultas	Agendamento, conflito de horário, convênio sem cobertura, carteirinha vencida, confirmação, remarcação com histórico e cancelamento	Aprovado
Ciclo da consulta	Encerramento automático 1 hora após o horário e registro do resultado pela recepção	Aprovado
Pagamentos	Cobrança só para particular; fluxo Stripe com Pix pendente e compensado; webhook repetido; assinatura inválida	Aprovado
Cadastros	Profissionais, especialidades, configurações, exclusão de médico sem consultas e desativação de paciente	Aprovado

Fonte: Elaborada pelos autores (2026).

5.5 Testes Automatizados

Os testes automatizados de integração foram escritos com o executor nativo do Node.js (`node:test`) e a biblioteca Supertest. Eles executam a API completa sobre um banco de dados de teste separado, com os serviços externos (Stripe e e-mail) simulados.

Cobertura de testes:

- 41 cenários automatizados cobrindo os fluxos principais: autenticação, permissões, agenda, consultas, pagamentos, cadastros e documentos médicos;
- Execução com o comando `npm test` a cada mudança relevante e antes da publicação de cada versão.

```
$ npm test
✓ recusa senha errada
✓ bloqueia rotas sem token
✓ paciente agenda para si e o horário sai da lista
✓ recusa horário já ocupado
✓ bloqueia convênio sem cobertura
✓ cobra pelo Stripe dentro do site e confirma pelo webhook
✓ encerra sozinha 1 hora após o horário e trava a consulta
...
ℹ tests 41   ℹ pass 41   ℹ fail 0
```

6 MANUAL DO USUÁRIO

Esta seção orienta o usuário final sobre como navegar e utilizar o sistema, explicando o funcionamento das principais telas, ações disponíveis e o que fazer em situações comuns. Idealmente, deve ser simples, objetiva e acessível até para pessoas não técnicas.

6.1 Guia de Navegação

Tela de acesso: o site possui uma tela de acesso única, com as abas “Login” e “Cadastre-se” e a opção “Esqueci minha senha”. Após o login, cada pessoa é direcionada para a área do seu perfil (paciente, recepção, profissional de saúde ou administrador).

Figura 9 – Tela de acesso do sistema



Fonte: Elaborada pelos desenvolvedores (2026).

Cadastro do paciente: na aba “Cadastre-se”, o paciente informa nome, CPF, telefone, e-mail, senha, data de nascimento e convênio. O CPF é validado e o e-mail não pode estar em uso.

Figura 10 – Tela de cadastro do paciente



Fonte: Elaborada pelos desenvolvedores (2026).

Área do Paciente:

- **Início:** próximas consultas com status e código, confirmação de presença, remarcação, cancelamento e histórico;
- **Agendar:** fluxo guiado por especialidade, médico, data e horário, com aviso de cobertura do convênio;
- **Notificações:** avisos da clínica, lembretes, atestados e receitas e pagamento da consulta particular;
- **Meu perfil:** dados cadastrais, contato e convênio.

Área da Recepção (menu principal):

- **Dashboard:** indicadores e atendimentos do dia;
- **Agenda:** agenda de todos os médicos, com horários livres, ocupados e bloqueados;
- **Consultas:** agendar, confirmar, remarcar, cancelar, enviar lembretes e registrar realizada ou não compareceu;
- **Pacientes, Fila de espera, Médicos, Horários e bloqueios** e **Notificações**.

Área do Profissional de saúde: consultas do dia, minha agenda, meus pacientes (histórico e emissão de atestado, receita ou pedido de exame), meus horários e notificações.

Área do Administrador: visão geral com indicadores e relatórios, usuários, médicos, especialidades, agendas e horários e configurações da clínica (convênios e valor da consulta particular).

Fluxo básico para o paciente:

a. Acessar o site da clínica e criar a conta em “Cadastre-se” (ou entrar, se já tiver conta).
b. Clicar em “Agendar” e escolher a especialidade e o médico.
c. Selecionar a data e um dos horários disponíveis e confirmar o agendamento.
d. Confirmar a presença quando receber o pedido de confirmação.
e. Comparecer à consulta e, se for particular, pagar pelo próprio site em “Notificações”.

6.2 FAQs e Problemas Comuns

Tabela 15 – FAQs e Problemas Comuns

Pergunta	Resposta
Não aparece nenhum horário para o médico que escolhi. O que faço?	Todos os horários daquela data estão ocupados ou bloqueados. Escolha outra data ou outro médico da mesma especialidade, ou peça à recepção para entrar na fila de espera.
Não consigo agendar uma especialidade.	Seu convênio pode não cobrir essa especialidade ou sua carteirinha pode estar vencida. Verifique em “Meu perfil” ou entre em contato com a clínica.
Esqueci minha senha.	Clique em “Esqueci minha senha”, informe o e-mail e o CPF e use o código de 6 dígitos enviado por e-mail (válido por 15 minutos).
Posso cancelar ou remarcar uma consulta?	Sim, antes do horário marcado. Após a remarcação, confirme novamente a presença na nova data.
Paguei com Pix ou boleto e aparece “aguardando confirmação”.	Pix e boleto podem levar algum tempo para serem compensados. Você será avisado por notificação quando o pagamento for confirmado.
Funciona no celular?	Sim, o sistema é responsivo e funciona em navegadores modernos de celular, tablet e computador.

Fonte: Elaborada pelos autores (2026).

7 MANUAL TÉCNICO

Esta seção orienta desenvolvedores e administradores sobre a estrutura do código-fonte, execução local, boas práticas de contribuição e documentação das APIs disponíveis. É essencial para manutenção, expansão e integração do sistema, caso ocorra algum problema e o usuário necessite de ajuda especializada.

7.1 Guia do Desenvolvedor

Estrutura de diretórios: o projeto é dividido em dois repositórios na organização `clinica-consulta` do GitHub.

```
/clinica-backend         → API REST (Node.js + Express + MongoDB)
├── src/
│   ├── app.js            → configuração do Express, middlewares e rotas
│   ├── server.js         → conexão com o banco e início do servidor
│   ├── config/           → conexão do Mongoose
│   ├── controllers/      → tratamento das requisições de cada recurso
│   ├── middlewares/      → autenticação, permissões e erros
│   ├── models/           → esquemas das coleções do MongoDB
│   ├── routes/           → rotas da API (/api/...)
│   ├── services/         → agenda, notificações, e-mail e pagamentos (Stripe)
│   ├── seed/             → preparação do banco e corpo clínico
│   └── utils/            → datas, CPF, textos e erros
├── tests/                → testes de integração
└── docs/                 → guia técnico de desenvolvimento e implantação

/clinica-frontend        → Interface web (React + TypeScript + Tailwind)
├── src/routes/           → telas de cada perfil (paciente, recepção, profissional,
administrador)
├── src/components/       → componentes reutilizáveis
└── README.md             → instruções básicas do projeto
```

7.1.1 Guia de Instalação

Pré-requisitos:

- Node.js 20 ou superior e npm;
- MongoDB local ou conta no MongoDB Atlas;
- Git instalado;
- IDE recomendada: Visual Studio Code.

Passos de instalação:

1. Clonar os repositórios:

```
git clone https://github.com/clinica-consulta/clinica-backend.git
git clone https://github.com/clinica-consulta/clinica-frontend.git
```

2. Configurar o back-end (arquivo `.env` na pasta `clinica-backend`):

```
PORT=3000
MONGODB_URI=mongodb://localhost:27017/gestao-consultas
JWT_SECRET=troque-por-um-segredo-forte
JWT_EXPIRES_IN=8h
CORS_ORIGIN=http://localhost:8080
FRONTEND_URL=http://localhost:8080
# opcionais: STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, BREVO_API_KEY
```

3. Instalar as dependências, preparar o banco e executar a API:

```
cd clinica-backend
npm install
npm run setup      # prepara o banco com os acessos iniciais e o corpo clínico
npm run dev        # API em http://localhost:3000
```

4. Executar o front-end:

```
cd clinica-frontend
npm install
npm run dev        # site em http://localhost:8080
```

Se a API estiver em outro endereço, ajuste a variável `VITE_API_URL` no arquivo `.env` do front-end. Sem as chaves do Stripe, o pagamento funciona em modo simulado para demonstração.

7.2 Documentação da API

Todas as rotas ficam sob o prefixo `/api` e trocam dados em JSON. São públicas as rotas de login, cadastro, recuperação de senha, as listas de especialidades e convênios e as rotas destinadas ao gateway de pagamento. As demais exigem o cabeçalho `Authorization: Bearer <token>`.

Tabela 16 – Documentação da API

Método	Endpoint	Descrição	Autenticação
POST	/api/auth/login	Realiza a autenticação do usuário	Não
POST	/api/auth/cadastro	Cadastro público do paciente	Não
GET	/api/especialidades	Lista as especialidades da clínica	Não
GET	/api/pacientes	Lista os pacientes (recepção e administrador)	Sim
GET	/api/profissionais	Lista o corpo clínico	Sim
GET	/api/consultas	Lista as consultas permitidas ao usuário logado	Sim
POST	/api/consultas	Agenda uma nova consulta	Sim
PATCH	/api/consultas/:id/status	Confirma, cancela ou registra o resultado da consulta	Sim

POST	/api/consultas/:id/pagamento/checkout	Abre a sessão de pagamento do Stripe	Sim
POST	/api/pagamentos/webhook	Recebe os eventos do Stripe (assinatura verificada)	Assinatura
GET	/api/fila-espera	Lista a fila de espera	Sim
GET	/api/notificacoes	Lista as notificações do usuário	Sim
GET	/api/relatorios/resumo	Indicadores do painel	Sim

Fonte: Elaborada pelos autores (2026).

7.2.1 Formato de exemplo de request

```
POST /api/consultas
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json

{
  "profissional": "6650f1c2a9e4b21d3c8e1a77",
  "especialidade": "Cardiologia",
  "data": "2026-11-18",
  "hora": "09:30",
  "observacoes": "Primeira consulta"
}
```

7.2.2 Resposta esperada

```
HTTP/1.1 201 Created
{
  "id": "6651a0d4c1b2e3f4a5b6c7d8",
  "codigo": "CON-0142",
  "data": "2026-11-18",
  "hora": "09:30",
  "especialidade": "Cardiologia",
  "status": "Agendada",
  "respostaConfirmacao": "Sem resposta",
  "canal": "Paciente"
}
```

7.3 Padrões de Código

7.3.1 Backend – JavaScript (Node.js + Express)

- Modelos em PascalCase: `Consulta`, `Paciente`, `FilaEspera`;
- Variáveis e funções em camelCase: `dataConsulta`, `horariosLivres`;
- Arquivos por recurso: `consultaController.js`, `consultaRoutes.js`;
- Organização por camadas: `/routes`, `/controllers`, `/services`, `/models`.

7.3.2 Frontend – React + TypeScript + Tailwind

- Componentes React em PascalCase: `AgendamentoForm.tsx`;
- Rotas por perfil: `paciente.agendar.tsx`, `profissional.agenda.tsx`;
- Separação de responsabilidades (componentes, páginas, serviços de API e estilos).

7.3.3 Boas práticas gerais

- Código comentado sempre que necessário;
- Funções curtas, com nomes descritivos;
- Commits no formato `tipo: descrição` (`feat`, `fix`, `docs`, `test`, `chore`);
- Segredos e chaves somente em variáveis de ambiente, nunca no repositório.

8 PROPOSTA COMERCIAL

8.1 Descrição da Proposta

A presente proposta tem como objetivo oferecer uma solução tecnológica simplificada para o agendamento e a gestão de consultas em clínicas médicas, por meio do desenvolvimento da plataforma Gestão de Consultas. Trata-se de um sistema personalizado, desenvolvido sob medida, com foco na usabilidade, acessibilidade, eficiência operacional e praticidade tanto para os pacientes quanto para a equipe da clínica.

A plataforma será composta por três camadas principais:

- **Frontend (interface do paciente e da equipe):** desenvolvido em React, TypeScript e Tailwind CSS, com foco em uma navegação simples e intuitiva, acessível via navegador em diferentes dispositivos;
- **Back-end/API:** desenvolvido em Node.js com o framework Express, responsável por toda a lógica de negócios, autenticação, permissões por perfil, regras de agendamento, notificações e integração com o gateway de pagamento;
- **Banco de Dados:** utilização do MongoDB Atlas, responsável pelo armazenamento seguro e estruturado das informações de pacientes, profissionais, convênios e consultas.

O sistema será projetado para operar de forma integrada à internet, com hospedagem em nuvem, sendo ideal para clínicas que desejam modernizar o atendimento, reduzir faltas, otimizar o controle da agenda e oferecer maior praticidade aos pacientes.

8.2 Escopo Comercial

8.2.1 Itens Inclusos

- Desenvolvimento das áreas de paciente, recepção, profissional de saúde e administrador;
- Integração com o Stripe para pagamento on-line da consulta particular;
- Publicação em nuvem (Render, Vercel e MongoDB Atlas);
- Workshops de todas as funcionalidades (até 2 sessões);
- Documentação completa (usuário, técnico e implantação);
- Treinamento remoto de até 4 horas.

8.2.2 Itens Exclusos

- Taxas cobradas pelo Stripe sobre cada transação;
- Integração com operadoras de convênio e prontuário eletrônico completo;
- Manutenção após os 3 meses de garantia.

8.2.3 Preço e Condições de Pagamento

Tabela 17 – Preço e Condições de Pagamento

Item	Descrição	Qtd/Horas	Valor Unitário (R$)	Total (R$)
1. Mão de Obra				
1.1	Analista de Requisitos (levantamento, documentação funcional)	40h	120,00	4.800,00
1.2	Desenvolvedor Back-end (Node.js/Express – API e regras de negócio)	100h	150,00	15.000,00
1.3	Desenvolvedor Front-end (React/TypeScript – telas responsivas)	90h	110,00	9.900,00
1.4	Analista de Testes e QA	30h	90,00	2.700,00
1.5	Gerente de Projeto (planejamento, reuniões e acompanhamento)	30h	130,00	3.900,00
	Subtotal Mão de Obra			**36.300,00**
2. Infraestrutura (12 meses)				
2.1	Hospedagem da API (Render)	12 meses	40,00	480,00
2.2	Banco de Dados Gerenciado (MongoDB Atlas)	12 meses	60,00	720,00
2.3	Hospedagem da interface web (Vercel Pro)	12 meses	110,00	1.320,00
2.4	Domínio Personalizado (.com.br)	1 ano	40,00	40,00
2.5	Serviço de E-mail Transacional (Brevo)	12 meses	50,00	600,00
	Subtotal Infraestrutura			**3.160,00**
3. Software				
3.1	Licença Figma Professional (Design)	3 meses	100,00	300,00
3.2	GitHub Team (5 usuários)	6 meses	125,00	750,00
3.3	Ferramenta de Testes de API (Postman Basic)	6 meses	75,00	450,00
	Subtotal Software			**1.500,00**

Fonte: Elaborada pelos autores (2026).

Total Geral do Projeto GESTÃO DE CONSULTAS

Mão de obra: R$ 36.300,00

Infraestrutura: R$ 3.160,00

Software: R$ 1.500,00

Valor Total: R$ 40.960,00

Condições de pagamento: 30% na assinatura do contrato, 40% na entrega da versão de testes e 30% na entrega final, por Pix, transferência ou boleto.

8.3 Cronograma de Entrega

Tabela 18 – Cronograma de Entrega

Fase	Duração estimada	Data de Início	Data de Término
Planejamento e Kick-off	1 semana	03/08/2026	07/08/2026
Desenvolvimento (6 ciclos de 2 semanas)	12 semanas	10/08/2026	30/10/2026
Testes e Ajustes Finais	2 semanas	02/11/2026	13/11/2026
Treinamento e Entrega	1 semana	16/11/2026	20/11/2026

Fonte: Elaborada pelos autores (2026).

8.4 Termos e Condições

- A clínica terá garantia de 3 meses para correções de bugs sem custo adicional;
- Qualquer customização além do escopo será cobrada à parte, em contrato suplementar;
- Todo o código-fonte desenvolvido será de propriedade da clínica, mediante quitação integral;
- As partes assinam acordo de confidencialidade (NDA) para proteger informações sensíveis;
- O tratamento dos dados pessoais e de saúde dos pacientes seguirá a LGPD (Lei nº 13.709/2018).

8.5 Validade da Proposta

Esta proposta é válida por 30 dias a partir da data de envio (até 06 de setembro de 2026). Após esse prazo, valores e prazos poderão ser revistos.

9 CONSIDERAÇÕES FINAIS

O desenvolvimento da plataforma Gestão de Consultas demonstrou a importância da integração entre tecnologia e gestão na área da saúde. A criação de um sistema de agendamento on-line permitiu centralizar a agenda dos profissionais, o cadastro de pacientes e convênios, a comunicação e a cobrança em uma única ferramenta, apoiando a tomada de decisão e a organização do fluxo de trabalho da clínica. Com isso, é possível reduzir as faltas, evitar conflitos de horário, reaproveitar horários cancelados por meio da fila de espera e garantir um atendimento mais eficiente e profissional.

A plataforma web desenvolvida apresenta uma interface intuitiva e acessível, com uma área específica para cada perfil, facilitando o agendamento de consultas de forma autônoma pelo paciente, inclusive por pessoas idosas. Além disso, a confirmação de presença, os lembretes e as notificações contribuem para o relacionamento com o paciente, e o pagamento on-line pelo Stripe permite encerrar o atendimento particular dentro da própria plataforma, sem que os dados de pagamento passem pelo servidor da clínica.

Os requisitos funcionais e não funcionais definidos durante o projeto asseguraram que a plataforma não apenas atendesse às necessidades de agendamento e gestão, mas também garantisse segurança, privacidade conforme a LGPD, confiabilidade e escalabilidade. A organização do código em camadas, os testes automatizados e a documentação detalhada permitem futuras melhorias, como relatórios por profissional com exportação, lembrete automático na véspera da consulta, prazo mínimo configurável para cancelamento, exportação dos dados do paciente e documentação interativa da API.

Por fim, este trabalho evidencia o potencial da tecnologia para transformar serviços tradicionais de saúde, mostrando que soluções digitais podem tornar o dia a dia de uma clínica mais eficiente e organizado. A plataforma desenvolvida serve como um modelo que pode ser replicado e aprimorado em outras clínicas e consultórios, contribuindo para a modernização do atendimento e para a gestão baseada em dados concretos.

Conclui-se, portanto, que a plataforma desenvolvida apresenta potencial significativo para apoiar a digitalização de clínicas médicas, promovendo maior controle sobre os agendamentos, redução das faltas, otimização do tempo da recepção e melhoria na experiência do paciente, ao mesmo tempo em que reforça o compromisso com a inovação e a qualidade no atendimento.

REFERÊNCIAS

ALMEIDA, Maria C. de. **Engenharia de Software Moderna**. 2. ed. São Paulo: Atlas, 2020.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR ISO/IEC 25010:2011**: Sistemas e software de engenharia – Requisitos e avaliação da qualidade de produto de software – Modelo de qualidade de produto. Rio de Janeiro: ABNT, 2011.

BEZERRA, Eduardo P. **Princípios de Análise e Projeto de Sistemas com UML**. 3. ed. Rio de Janeiro: LTC, 2015.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 10 ago. 2026.

DRIESSEN, Vincent. **A successful Git branching model**. 2010. Disponível em: https://nvie.com/posts/a-successful-git-branching-model/. Acesso em: 10 ago. 2026.

EXPRESS. **Documentação oficial do Express**. Disponível em: https://expressjs.com/. Acesso em: 10 ago. 2026.

FIELDING, Roy Thomas. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Ciência da Informação e da Computação) – University of California, Irvine, 2000.

JONES, M.; BRADLEY, J.; SAKIMURA, N. **RFC 7519**: JSON Web Token (JWT). IETF, 2015. Disponível em: https://datatracker.ietf.org/doc/html/rfc7519. Acesso em: 10 ago. 2026.

MONGODB. **Documentação oficial do MongoDB**. Disponível em: https://www.mongodb.com/docs/. Acesso em: 10 ago. 2026.

NODE.JS. **Documentação oficial do Node.js**. Disponível em: https://nodejs.org/docs/. Acesso em: 10 ago. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software**: uma abordagem profissional. 8. ed. Porto Alegre: McGraw-Hill, 2016.

REACT. **Documentação oficial do React**. Disponível em: https://react.dev. Acesso em: 10 ago. 2026.

SCHWABER, Ken; SUTHERLAND, Jeff. **The Scrum Guide**. 2020. Disponível em: https://scrumguides.org. Acesso em: 10 ago. 2026.

STRIPE. **Documentação oficial do Stripe**: Embedded Checkout e webhooks. Disponível em: https://docs.stripe.com/. Acesso em: 10 ago. 2026.

TAILWIND CSS. **Documentação oficial do Tailwind CSS**. Disponível em: https://tailwindcss.com/docs. Acesso em: 10 ago. 2026.